

Лаборатория протоколов

Исследование и описание неизвестного сетевого протокола
поверх TCP

Команда HTTP 418 · UMIRHack 2026 · кейс #02 · Мадригал
Python 3.12 · PySide6 · dpkt · pytest · Docker



Состав команды

HTTP 418 · UMIRHack 2026 · кейс #02 ·

Мадригал

Сабадаш Павел — архитектура, capture, project, report, координация, backend, GUI

Предков Никита — backend, правила, тесты

github.com/PavelI778/madrigal-protocol-lab

Задача: восстановить протокол, а не угадать его

Дано: захваты (pcap/pcapng) и журнал действий устройства. Спецификации нет.

Нужно: проверяемая интерпретация структуры сообщений.

Отличие от «просто распарсить»: парсер даёт один ответ; здесь видно, где он верен, а где перестаёт работать.

2

входа: байты + журнал

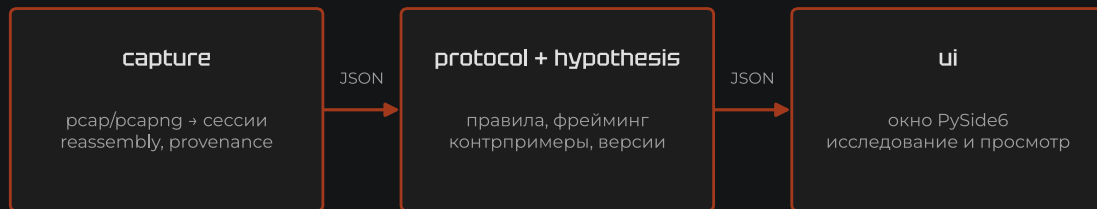
3

вопроса: что наблюдалось · где работает · почему это чтение

01

Πορχογ

Три модуля, связанные JSON-контрактами



Контракты зафиксированы JSON Schema (draft 2020-12)

capture

pcap/pcapng →

protocol + hypothesis

фрейминг, поля,

ui

настольное окно

02

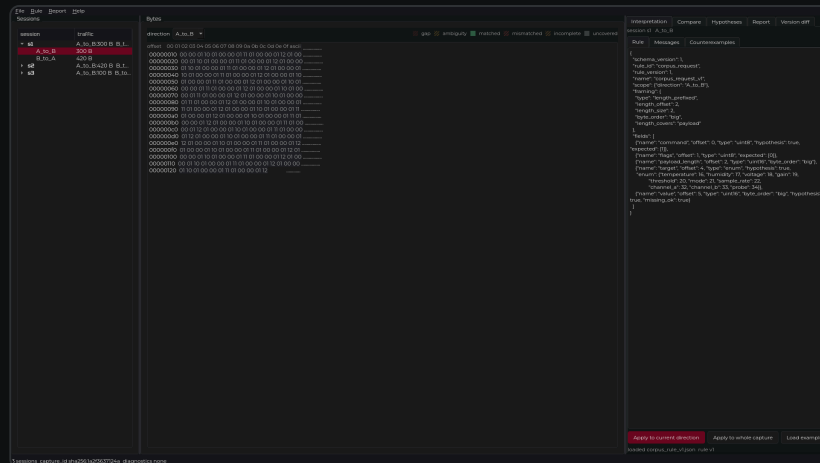
Реализация

Захват потока: границы ТСП ≠ границы сообщений

Сессия — экземпляр соединения, а не пара адресов: повторное использование портов даёт новую сессию.

Байты восстанавливаются по номерам последовательности; повторная передача даёт сведения о происхождении, а не лишние байты.

Пропуск не заполняется нулями — становится диагностикой,



Правила и фрейминг: одно описание на весь поток

Фрейминг: по длине, фиксированный размер, по маркерам, вручную.

Типы полей: `uint` `bytes` `enum`
`string` `checksum` `computed`

Поле с пометкой `hypothesis` — предположение о смысле, а не факт.

Сообщения не перечисляются вручную: правило описывает весь поток.

```
{
  "framing": {
    "type": "length_prefixed",
    "length_offset": 2, "length_size": 2,
    "byte_order": "big", "length_covers": "payload"
  },
  "fields": [
    {
      "name": "command", "offset": 0,
      "type": "uint8", "hypothesis": true,
      "expected": [1]
    }
  ]
}
```

Гипотезы и контрпримеры: совпадение – не доказательство

Шесть статусов по каждому сообщению:

■ **matched** · ■ **mismatched** · ■ **ambiguous**

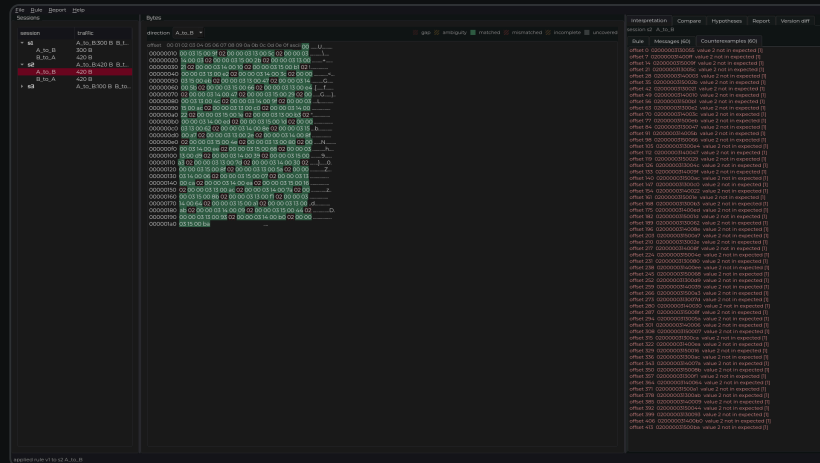
incomplete

uncovered

not_applicable

mismatched — это контрпример, привязанный к байтам и к пакету-источнику.

Правило проверяется на **всём** корпусе.



03

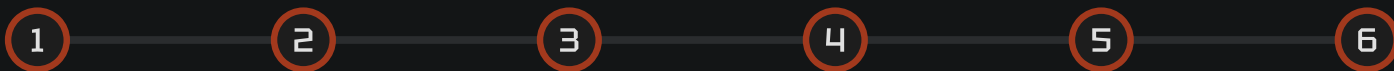
Демонстрация

Цикл на корпусе: шесть шагов

наблюдение

проверка

уточнение



гипотеза

контрпример

повторная
проверка

Наблюдение: байты запроса начинаются с `01 00 00 01`.

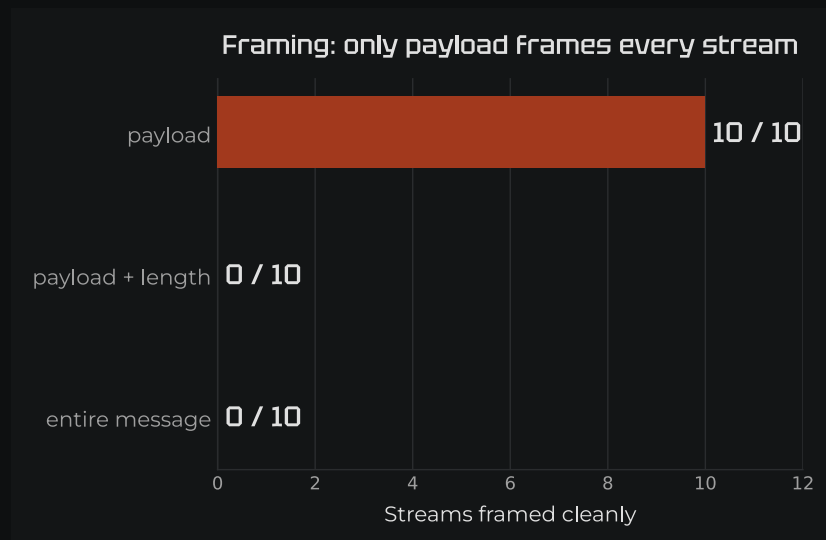
Гипотеза: по смещению 2 — длина два байта, старший вперёд.

Precision: 0.43 → 1.00

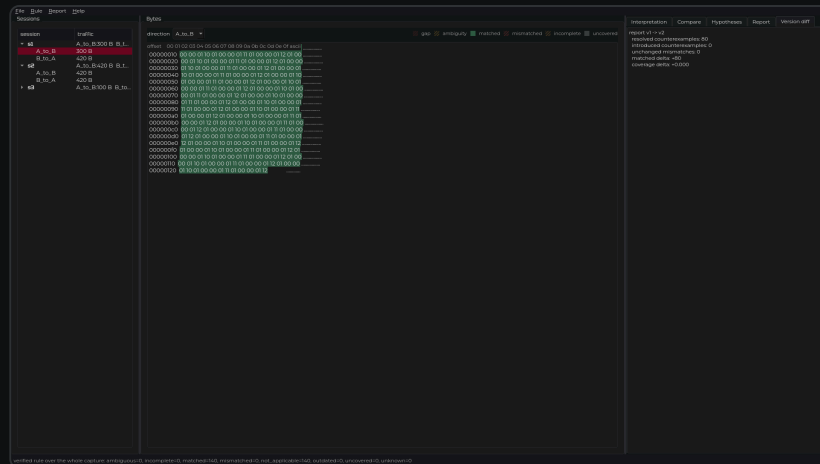
Version	Precision
v1	0.43
v2	1.00

показатель	v1	v2
matched	60	140

Сравнение версий: изменён один список, структура та же



Фрейминг `payload` разбирает 10/10 потоков; два других кандидата — 0/10.



Изменён **только** список

допустимых команд: **[1]** → **[1, 2, 3]**.

Контрпример как ценность: байт, который уточнил правило

Конкретный `mismatched`-байт:

поле	значение
session / direction	<code>s2</code> / <code>A_to_B</code>
message offset	<code>0</code> , length <code>7</code>
bytes	<code>02 00 00 03 13 00</code> <code>55</code>
reason	<code>value 2 not in expected [1]</code>

Как уточнили правило:

```
{ "name": "command", "offset": 0,  
  "type": "uint8", "hypothesis": true,  
-  "expected": [1] }  
+  "expected": [1, 2, 3] }
```

Один байт, привязанный к пакету-источнику, превратил 80 контрпримеров в 0 (v1 → v2).

Метрики: сводка чисел

140

matched, rule v2

0

counterexamples, v2

10/10

потоков разобрано

280

сообщений в
захвате 01

40/40

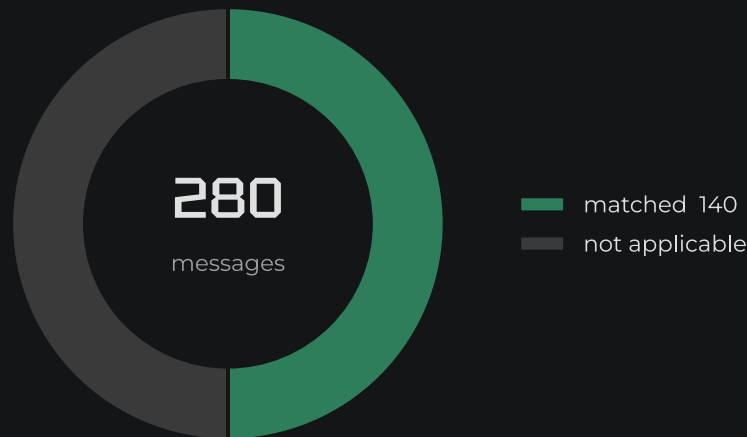
перенос на захват
02

5/5

CI зелёный

Источник: `docs/METRICS.md`, `docs/AUDIT.md`.

Coverage of corpus_capture_01 (rule v2)



04

Итог

Ограничения: честный список

ИТОГ И ССЫЛКИ

Вопросы

Команда HTTP 418 · UMIRHack 2026

github.com/Pavel1778/madrigal-protocol-lab